Human Pose Estimation in Sports - 3D Pose Reconstruction

Alessandro Rossi, Elia Santuari

University of Trento

Abstract. This project addresses multi-view 3D human pose estimation and player tracking on a basket court. The main objective is to reconstruct accurate 3D player skeletons from multi-camera videos. Starting from annotated 2D joint keypoints, initial 3D positions are estimated through multi-view triangulation. To mitigate annotation noise and camera distortion, Bundle Adjustment is applied to jointly refine the camera parameters and 3D joint locations. The overall system is evaluated using standard reprojection error metrics, demonstrating that global optimization significantly enhances 3D reconstruction accuracy.

1 Skeleton Annotation

The dataset provided for this project consists of multi-view recordings from a calibrated setup of six cameras (Fig. 1), where Camera 7 spans the entire court, introducing significant visual distortion. Our analysis focuses on our group's assigned action, comprising 5 frames across all cameras (30 views in total). To establish a reliable ground truth, we manually annotated player labels and joint positions using the Roboflow platform.

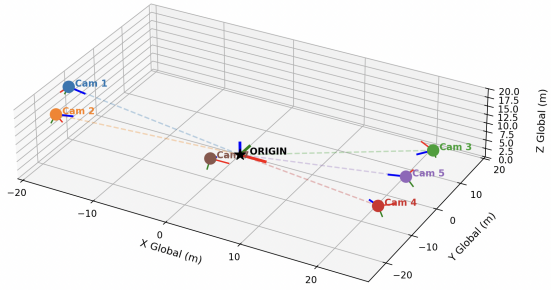


Fig. 1: Camera setup.

2 3D Player's Position

The aim of this section is to explain the workflow from the annotated player skeleton data to the 3D position reconstruction on the court, concluding with metric evaluations of the point reprojections.
The extrinsic and intrinsic parameters of each camera are given; specifically, the distortion coefficients d, the intrinsic matrix K, the rotation vector r_{vec} (from which we derive the rotation matrix R using Rodrigues' formula), and the translation vector t. From these, we obtain the projection matrix as [6]:

$$P = K[R|t]$$

Next, we extract the annotated skeleton data. For any given frame, camera, and player (e.g., 'Red_11'), the annotation consists of 54 values structured as 18 three-element tuples, one for each joint. Each tuple provides the 2D coordinates (x, y) and a visibility flag indicating whether the joint is visible (2), occluded (1), or out of frame (0).

2.1 Rectification

One important aspect to take always in consideration is that the cameras have some kind of optical distortion, radial and tangential, described with the parameters that we have just listed. With a process called rectification we can align image perspectives, in this way we project images onto a common image plane so that corresponding points across images lie on the exact same horizontal pixel row. It's very important to remember that not just images should be rectified but also the 2D points (annotations).

2.2 Triangulation

With the required parameters available, we can estimate the 3D joint positions via triangulation. Given 2D points x and projection matrices P, theoretical triangulation solves $x = PX$. However, due to noise and camera distortions, multi-view projection rays rarely intersect at a single point in 3D space. The task thus reduces to finding an optimal 3D coordinate X that minimizes the reprojection error across multiple views.
There are many different approaches in order to face this problem here is a list with a brief explanation of some [5, 6]:

- **Linear/Midpoint method**: they are very simple and fast methods to directly solve this problem, it could lead to larger errors but still work well.
- L_2 **and** L_∞ **normalization**: they are the mostly used techniques, they define a cost function and use an optimization algorithm to find its minimum, the cost function measures an error of the triangulated point with respect to the annotated 2D points.

Linear Method: here follows a brief explanation of how it mathematically works. Imagine to have $x_i = P_i X$ where x_i is the corresponding point of the 3D space point X in the $i-th$ camera plane and P_i is the camera matrix. Let's see who x_i and P_i are:

$$x_i = w \cdot \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}, \quad P_i = \begin{bmatrix} p_i^{1T} \\ p_i^{2T} \\ p_i^{3T} \end{bmatrix} \text{ with } P_i \in \mathbb{R}^{3\times 4}$$

From these two we can define a system of three equations:

$$\begin{cases} w \cdot u_i = p_i^{1T} \cdot X \\ w \cdot v_i = p_i^{2T} \cdot X \\ w = p_i^{3T} \cdot X \end{cases} \implies \begin{cases} u_i \cdot p_i^{3T} \cdot X - p_i^{1T} \cdot X = 0 \\ v_i \cdot p_i^{3T} \cdot X - p_i^{2T} \cdot X = 0 \end{cases}$$

With more than two cameras, the system becomes overdetermined. Because the equations are inconsistent due to noise, an exact solution does not exist; we thus solve a linear least-squares problem. Stacking the two linear equations for each available view yields the homogeneous system $AX = 0$. In order to solve this problem we have to minimize a linear system, the purpose is to find the minimum singular value of A. Here follows the formalization of the optimisation problem: $\min_{\mathbf{X}} \|\mathbf{AX}\| \quad \text{s.t.} \quad \|\mathbf{X}\| = 1$.
We can use Singular Value Decomposition (SVD) to solve it, $A = U\Sigma V^T$. The solution is the last column of V, i.e., the singular vector associated with the lowest singular value of Σ. In this way it will return a vector $X = (X_0, X_1, X_2, X_3)^T$ in homogeneous coordinates, in order to have the real ones we have to divide each element by the scaling X_3.

$$X_{\text{real}} = \left(\frac{X_0}{X_3}, \frac{X_1}{X_3}, \frac{X_2}{X_3}\right)^T$$

***L_2* Normalization:** There will always be a certain error between the position of the image point and the real one, the problem is that the real is unknown. We only know that the assumed real image point coordinates can only be obtained through the projection matrix. We know that $\sum_i d(x_i, \hat{x}_i)$ is the reprojection error, the distance between the 3D position reprojection from the triangulation algorithm $\hat{x}_i$ and the position of that point on the i-th image plane x_i. One can say, let's find the best 3D point in the real world that minimizes the reprojection error, let's then minimize the L_2-Norm:

$$\min_X \sum_i d(x_i, \hat{x}_i) = \min_X \sum_i ||x_i - P_i X|| = \min_X \sum_i ||x_i - \frac{p_i^{1:2} X}{p_i^3 X}||_p$$

Where $x_i = \binom{u_i}{v_i}$ and p is the p-norm, in our case 2. In order to solve the problem, a Non-Linear optimization algorithm should be used, with a Stochastic Gradient Descent method. In practice, we are trying to find the best 3D points in the real world, our parameters (X, Y, Z) that represent each joint of a certain skeleton, by minimizing a cost function that in our case is the squared of the distance between the 2D point in the image plane (x_i) and the 3D point (X) reprojected into the image plane through the matrix P. In the formula, $p_i^{1:2}$ represents the first two rows of the projection matrix and p_i^3 is the third.

NOTE: The results reported below refer exclusively to the linear method (SVD), as the L2 optimization yielded negligible improvement.

Qualitative visualization of the reconstructed 3D skeletons provides a crucial sanity check for spatial alignment and pose estimation. Figure 2 illustrates the full-court 3D reconstruction alongside detailed individual skeletons for players "White_10" (left) and "Red_11" (right). While non-quantitative, this visual comparison confirms that estimated player positions and skeletal poses remain physically plausible.

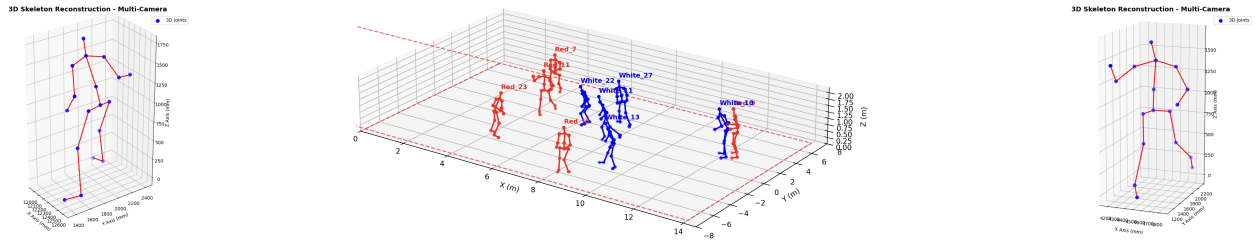


Fig. 2: Full-court 3D skeletons and detailed pose visualizations for players "White_10" (left) and "Red_11" (right).

If we just plot in a sequence the 3D pose of the players on the court we get a GIF of the entire action, with the movements of all the players. A video of this animation looped several times can be found on the git repository: https://github.com/Computer-Vision-UniTN/cv25-26-human-pose-estimation-alessandrorossi-eliasantuari.

2.3 Reprojection on the Image Plane

Now that we have reconstructed the 3D real world positions of each joint we want to reproject them into the camera view, in order to visualize how the reconstruction's gone. In this way we see the original image from each camera perspective with the players with their skeleton drawn on them. But since we have rectified the players pixel coordinates we also need to rectify the images on which we are going to reproject the joints, otherwise we applied a transformation to the points but not on the image. Given the 3D joint position X, to reproject in 2D, it is sufficient to:

$$x_i = P_i X = \begin{bmatrix} p_i^{1T} X \\ p_i^{2T} X \\ p_i^{3T} X \end{bmatrix} = \begin{bmatrix} w \cdot u_i \\ w \cdot v_i \\ w \end{bmatrix}$$

The result will be converted into 2D pixel coordinates (u_i, v_i) representing the position of each joint on the camera plane of that specific camera by dividing by the scale factor $w = p_i^{3T} X$:

$$u_i = \frac{p_i^{1T} X}{p_i^{3T} X}, \quad v_i = \frac{p_i^{2T} X}{p_i^{3T} X}$$



Fig. 3: Reprojected skeletons on the image plane of camera 3 and 4 respectively.

In the images in figure (3) we can already see that in some cases the reprojected skeletons fit well on the correspondent player, while in some others they don't, they are shifted or in other cases the joints and bones are not well aligned with the player's.

2.4 Estimation of the Triangulation Accuracy

In order to have a numerical idea of how the triangulation works, we may compare for each image and each frame the reprojected points with the ground truth (GT) from the coco annotations file, as mentioned above, we should rectify also these points. To compare the reprojected points we used three metrics:

- MPE[4]: computes the Euclidean distance pixel by pixel, imagine that $\hat{p}_i = (\hat{x}_i, \hat{y}_i)$ is the 2D i-th reprojected point and $p_i = (x_i, y_i)$ is the 2D ground truth i-th point. We can compute the reprojection error as:

$$\text{MPE} = \frac{1}{N} \sum_{i=1}^{N} \sqrt{(\hat{x}_i - x_i)^2 + (\hat{y}_i - y_i)^2}$$

  Where N is the number of joints per player.
- OKS [8, 4]: this metric takes into account the fact that a player closer to the camera will be penalised more than one further away with the MPE, this metrics compensates for this by considering the scale of the player.

$$\text{OKS} = \frac{\sum_i \exp(-d_i^2/(2s^2k_i^2)) \cdot v_i}{\sum_i v_i}$$

  Where d_i is the previously computed Euclidean distance, s is the player scale ($\sqrt{\text{bounding box area}}$), k_i is the joint tolerance constant, and v_i is the ground truth visibility flag.
  In this way we take into account also the distance of the players from the camera, thus they are not penalised if closer to cameras. K_i is avoiding that small points (like the head) are evaluated with the same tolerance of bigger points (like the spine). We use a big penalisation on smaller points and less severe on bigger points.
- To obtain a more intuitive and physical metric, we can convert the 2D pixel distance into real-world units using pinhole camera projection geometry. According to the pinhole camera model, the conversion scale from pixels to centimeters at a specific depth Z_i depends directly on the camera's focal length f_x:

$$\text{Scale}_i = \frac{Z_i}{10 \cdot f_x} \quad [\text{cm/pixel}]$$

  where the joint depth Z_i along the optical axis (in millimeters) is computed from the 3D joint in homogeneous coordinates $\mathbf{X}_i = [X_i, Y_i, Z_i, 1]^T$ using the third row of the camera projection matrix P_i, thus as $Z_i = \mathbf{p}_3^T \mathbf{X}_i$.
  To obtain the final result we just have to multiply the MPE by the scaling factor above.

3 Bundle Adjustment

Up to now, we used the initial calibration parameters provided. However, positioning errors and slight intrinsic inaccuracies can remain. To refine them, Bundle Adjustment (BA) performs a global optimization of both intrinsic and extrinsic camera parameters by minimizing the reprojection error between projected 3D points and their 2D ground-truth measurements [1, 2, 6, 7].
Bundle Adjustment requires known 3D reference points, such as court line intersections. We utilized calibration parameters from the "second version" (SV) alongside court reference points from both the "first version" (FV) and SV folders. For further details on data selection, consult the notebook `bundle_adjustment.ipynb`
The vector of 15 parameters to optimize for each camera is defined as:

$$\boldsymbol{\theta} = [r_x, r_y, r_z, t_x, t_y, t_z, f_x, f_y, c_x, c_y, k_1, k_2, p_1, p_2, k_3]^T$$

Where $\mathbf{r}$ and $\mathbf{t}$ denote rotation and translation, (f_x, f_y) are focal lengths, (c_x, c_y) is the principal point, and (k_1, k_2, k_3) and (p_1, p_2) represent radial and tangential distortion coefficients, respectively.
Let X_i be a 3D world point and x_i its corresponding 2D ground-truth pixel location. The non-linear camera model predicts the 2D position as $\hat{x}_i = \pi(X_i; \boldsymbol{\theta})$. Therefore we minimize the L_2-norm of the residual vector $r_i = \hat{x}_i - x_i$:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \frac{1}{2} \sum_{i=1}^{N} \|\pi(X_i; \boldsymbol{\theta}) - x_i\|^2 = \arg\min_{\boldsymbol{\theta}} \frac{1}{2} \|\mathbf{r}(\boldsymbol{\theta})\|_2^2$$

Where $\pi(X_i; \boldsymbol{\theta})$ represents the projection function evaluated at the current parameter estimate $\boldsymbol{\theta}$.

In the multi-camera optimization pipeline, we employ the Trust Region Reflective (TRF) solver. While the Levenberg-Marquardt algorithm is widely used in standard calibration literature, TRF was selected specifically for its native support of parameter bounds ($\boldsymbol{\theta}_{lb} \leq \boldsymbol{\theta} \leq \boldsymbol{\theta}_{ub}$). The optimization is formulated as a bounded non-linear least-squares problem:

$$\min_{\boldsymbol{\theta}_{lb} \leq \boldsymbol{\theta} \leq \boldsymbol{\theta}_{ub}} \frac{1}{2} \|\mathbf{r}(\boldsymbol{\theta})\|_2^2$$

Imposing explicit bounds around the initial parameters $\boldsymbol{\theta}_0$ prevents the optimizer from overfitting. Otherwise, the optimization may yield parameters that perfectly fit the court reference points but perform poorly on the skeletons.



Fig. 4: Ref. points before and after BA (left) and player skeletons before (blue/red) and after (purple/orange) BA (right).

As shown in Fig. 4 (Camera 5), skeleton alignment improved across most views. Despite minor localized exceptions, the overall reprojection error was successfully reduced, as detailed in the next section.

4 Results

This section presents the final evaluation of our pipeline, as previously said the results reported refer exclusively to the linear method (SVD), as the L2 optimization yielded negligible improvements. Table 1 reports the Mean Reprojection Error (MPE) in centimeters across all frames for each player and camera view. Before Bundle Adjustment (BA), the overall mean error was 12.27 cm, establishing an acceptable baseline. Applying BA significantly improved calibration accuracy, reducing the overall mean error to 8.72 cm. Higher localized errors (e.g., Camera 7 or specific players) are primarily attributable to 2D keypoint annotation errors and severe inter-player occlusions in those views. For the other previously defined metrics, please refer to the `project.ipynb` notebook. We also recommend using the provided interactive widgets to better visualize the reprojections and 3D skeletons.

(a) Before Bundle Adjustment

Player	C1	C2	C3	C4	C5	C7	Mean
R11	12.90	13.17	16.15	10.36	14.15	17.73	14.18
R13	10.43	—	7.50	—	27.61	28.65	18.55
R23	11.21	—	10.96	—	20.82	20.17	15.79
R7	5.05	3.74	—	5.61	17.17	6.15	7.55
R9	10.46	—	7.65	6.07	13.45	36.31	14.79
W10	9.68	—	6.91	5.66	10.23	19.49	10.39
W11	10.61	—	7.41	5.45	10.28	13.08	9.37
W13	9.00	—	9.12	4.26	20.98	19.65	12.60
W22	10.14	—	9.91	7.24	10.48	22.84	12.12
W27	6.89	8.72	12.17	7.40	13.68	9.09	9.66
Mean	**9.64**	**7.39**	**9.76**	**6.51**	**15.88**	**19.31**	**12.27**

(b) After Bundle Adjustment

Player	C1	C2	C3	C4	C5	C7	Mean
R11	10.35	9.89	10.09	8.39	15.27	18.11	12.25
R13	4.28	—	7.12	—	7.24	19.11	9.44
R23	4.01	—	6.97	—	6.81	19.29	9.27
R7	3.23	3.38	—	3.97	9.16	5.16	4.98
R9	7.65	—	5.71	4.09	9.95	31.29	11.74
W10	6.37	—	4.40	4.30	8.34	12.82	7.24
W11	7.42	—	6.27	4.87	10.24	13.25	8.41
W13	6.11	—	8.06	4.22	7.65	17.57	8.72
W22	6.10	—	6.34	5.50	10.49	23.52	10.39
W27	3.93	4.83	4.18	4.72	5.37	9.38	5.40
Mean	**5.95**	**5.07**	**6.57**	**5.01**	**9.05**	**16.95**	**8.72**

Table 1: Overall Mean Reprojection Error (MPE in cm) per player and camera before and after Bundle Adjustment.

5 Conclusions

This project provided a highly valuable learning opportunity, offering practical, hands-on experience in translating theoretical computer vision concepts into a functional, real-world application. We successfully developed an end-to-end pipeline for multi-view 3D human pose reconstruction on a basketball court, navigating the practical complexities of multi-camera triangulation.

Crucially, our implementation demonstrated that while linear triangulation methods (SVD) establish a reasonable starting point, real-world noise and lens distortions necessitate robust global optimization. By applying Bundle Adjustment, we effectively refined both the camera parameters and the 3D joint estimates, achieving a significant reduction in the overall mean reprojection error from 12.27 cm to 8.72 cm. Ultimately, this project highlighted both the power and the challenges of multi-view geometry in sports analytics. Future improvements could focus on integrating automated deep-learning keypoint detectors to replace manual annotations and employing advanced filtering techniques to handle severe inter-player occlusions.

References

1. Bill Triggs, Philip F. McLauchlan, Richard I. Hartley, and Andrew W. Fitzgibbon. *Bundle Adjustment — A Modern Synthesis*. In Vision Algorithms: Theory and Practice, Springer, 1999.
2. Yu Chen, Yisong Chen, and Guoping Wang. *Bundle Adjustment Revisited*. In Advances in Visual Computing, 2010.
3. David A. Forsyth and Jean Ponce. *Computer Vision: A Modern Approach*. Pearson, 2nd edition, 2011.
4. Shradha Dubey and Manish Dixit. *A comprehensive survey on human pose estimation approaches*. Multimedia Systems, 28(5), 2022.
5. Jia Chen et al. *Multi-View Triangulation: Systematic Comparison and an Improved Method*. IEEE Access, 2021.
6. Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd edition, 2004.
7. Tianjian Jiang et al. *WorldPose: A World Cup Dataset for Global 3D Human Pose Estimation*. In Proceedings of the European Conference on Computer Vision (ECCV), 2024.
8. COCO Consortium. *COCO Dataset - Keypoints Evaluation*. Available online: https://cocodataset.org/#keypoints-eval.